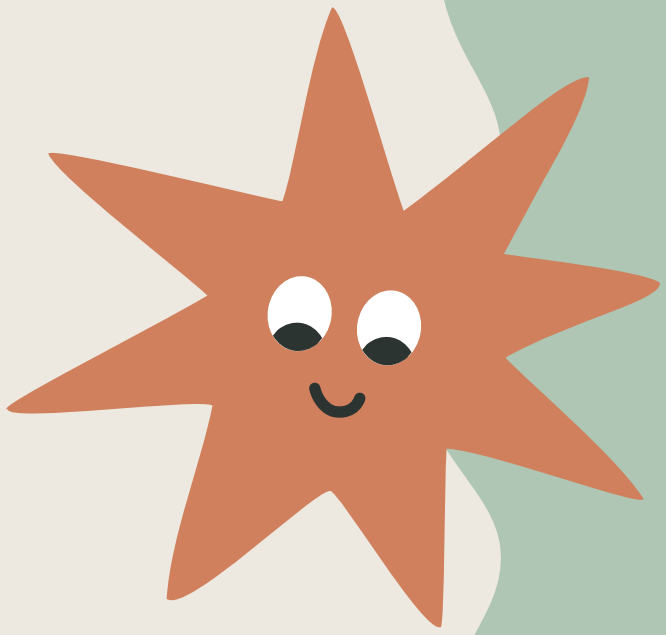
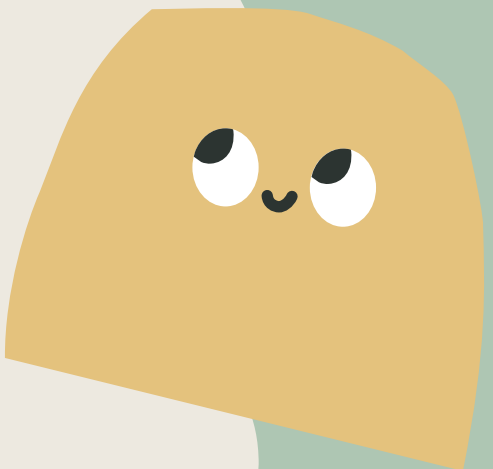
# Lists in Python

Lecture 7





# Learning Objectives

➢By the end of this lecture, students should be able to:

- Understand what a list is and why lists are useful
- Create lists in Python
- Access list elements using indexing
- Extract elements using slicing
- Add elements to a list
- Remove elements from a list
- Use important list methods
- Loop through lists
- Apply lists to practical programming and AI/data-processing examples

Prepared By: Ehtisham Rasheed

# What is a List?

- A **list** is an ordered collection of elements.
- Example:

```
students = ["Ali", "Ahmed", "Sara", "Ayesha"]
```

- Lists are:
  - Ordered
  - Changeable
  - Able to contain multiple values
  - Able to contain different data types
  - Allow duplicate values

Prepared By: Ehtisham Rasheed

# Creating a List

- Lists are created using square brackets:
- Example:

```
numbers = [10, 20, 30, 40, 50]

print(numbers)
```

- Output:

```
[10, 20, 30, 40, 50]
```

Prepared By Rasham Rasheed

# Different Types of Lists

- List of integers

```
marks = [78, 85, 91, 66, 72]
```

- List of strings

```
students = ["Ali", "Ahmed", "Sara"]
```

- Lists of floats

```
temperatures = [25.5, 27.3, 31.8, 29.6]
```

- List of Boolean values

```
results = [True, False, True, True]
```

Prepared by: Ehtisham Rasheed

# Different Types of Lists

- Python lists can contain different types of values.

```
student = ["Ali", 20, 3.45, True]
```

- We can create an empty list:

```
students = []
```

- Later, we can add elements to it

```
students.append("Ali")
students.append("Ahmed")

print(students)
```

- We can use len() to find the number of elements.

```
print(len(students))
```

Prepared by Ihtisham Rasheed

# List Indexing

- Lists use **indexing** to access individual elements.
- Consider:

```
students = ["Ali", "Ahmed", "Sara", "Ayesha"]
```

- The indexes are:

```
Positive:   0       1       2       3
          Ali     Ahmed    Sara    Ayesha
Negative:  -4      -3       -2      -1
```

- Accessing elements:

```
print(students[0])
print(students[1])
print(students[2])
print(students[3])
```

```
print(students[-1])
print(students[-2])
print(students[-3])
print(students[-4])
```

Prepared By: Eng. Rasheed

# Updating a List Element

- Lists are **mutable**, which means we can change their elements.
- Example:

```
students = ["Ali", "Ahmed", "Sara"]

students[1] = "Usman"

print(students)
```

- Output:

```
['Ali', 'Usman', 'Sara']
```

Prepared By: Rasheed

# List Slicing

➢Slicing allows us to extract **multiple elements** from a list.

```
numbers = [10, 20, 30, 40, 50]

print(numbers[1:4])
```

```
[20, 30, 40]
```

➢More examples:

```
print(numbers[:3])
```

```
[10, 20, 30]
```

```
print(numbers[2:])
```

```
[30, 40, 50]
```

```
print(numbers[0:6:2])
```

```
[10, 30, 50]
```

➢This command print the entire list

```
print(numbers[:])
```

Prepared By: Ehtisham Rasheed

# Adding Elements

➢The append() method adds an element to the **end** of a list.

```
students = ["Ali", "Ahmed"]

students.append("Sara")

print(students)
```

```
['Ali', 'Ahmed', 'Sara']
```

➢extend() adds multiple elements to a list.

```
students = ["Ali", "Ahmed"]

students.extend(["Sara", "Ayesha"])

print(students)
```

```
['Ali', 'Ahmed', 'Sara', 'Ayesha']
```

Rasheed By: Ehtisha

# append() vs extend()

```
numbers = [1, 2]

numbers.append([3, 4])

print(numbers)
```

```
[1, 2, [3, 4]]
```

```
numbers = [1, 2]

numbers.extend([3, 4])

print(numbers)
```

```
[1, 2, 3, 4]
```

Property: Ehtisham Rasheed

# Adding at a Specific Position — insert()

➢Use insert() when you want to add an element at a particular index.

```
students = ["Ali", "Sara"]

students.insert(1, "Ahmed")

print(students)
```

```
['Ali', 'Ahmed', 'Sara']
```

Prepared By Mutisham Rasheed

# Removing Elements — remove()

➢remove() removes an element by its value.

```
students = ["Ali", "Ahmed", "Sara"]

students.remove("Ahmed")

print(students)
```

```
['Ali', 'Sara']
```

➢If the value does not exist, remove() produces an error.

Prepared By Ntisham Rasheed

# Removing the Last Element — pop()

- pop() removes and returns an element.

```
students = ["Ali", "Ahmed", "Sara"]

removed = students.pop()

print("Removed:", removed)
print(students)
```

```
Removed: Sara
['Ali', 'Ahmed']
```

- We can specify which element to remove

```
students = ["Ali", "Ahmed", "Sara"]

removed = students.pop(1)

print("Removed:", removed)
print(students)
```

```
Removed: Ahmed
['Ali', 'Sara']
```

Ehtisham Rasheed

# Removing All Elements — clear()

➢The clear() method removes everything.

```
students = ["Ali", "Ahmed", "Sara"]

students.clear()

print(students)
```

```
[]
```

Prepared by Mohammad Basham Rasheed

# Important List Methods

| Method | Purpose |
| --- | --- |
| `append()` | Add one element at the end |
| `extend()` | Add multiple elements |
| `insert()` | Add at a specific index |
| `remove()` | Remove by value |
| `pop()` | Remove by index / last item |
| `clear()` | Remove all elements |
| `sort()` | Sort the list |
| `reverse()` | Reverse the list |
| `count()` | Count occurrences |
| `index()` | Find the index of a value |

# Sorting in Specific Order

- We can sort a list in ascending order.

```
numbers = [50, 10, 40, 20, 30]

numbers.sort()

print(numbers)
```

```
[10, 20, 30, 40, 50]
```

- Sorting in descending order

```
numbers = [50, 10, 40, 20, 30]

numbers.sort(reverse=True)

print(numbers)
```

```
[50, 40, 30, 20, 10]
```

Rasheed By: Ehtisham

# Other Functions

- The reverse() method reverses the current order.

```
numbers = [10, 20, 30, 40]

numbers.reverse()

print(numbers)
```

```
[40, 30, 20, 10]
```

- count() tells us how many times a value appears.

```
numbers = [10, 20, 10, 30, 10, 40]

print(numbers.count(10))
```

```
3
```

- index() finds the position of the first occurrence of a value.

```
students = ["Ali", "Ahmed", "Sara", "Ahmed"]

print(students.index("Ahmed"))
```

```
1
```

# Checking Whether an Element Exists

- We can use the in operator.

```
students = ["Ali", "Ahmed", "Sara"]

print("Ali" in students)
print("Usman" in students)
```

```
True
False
```

- Looping through list

```
students = ["Ali", "Ahmed", "Sara", "Ayesha"]

for student in students:
    print(student)
```

```
Ali
Ahmed
Sara
Ayesha
```

Prepared By: Ehtisham Rasheed

# Find the Highest & Lowest Value

- We can use the built-in max() function:

```
marks = [78, 85, 91, 66, 72]

highest = max(marks)

print("Highest:", highest)
```

```
Highest: 91
```

- We can use the built-in min() function:

```
marks = [78, 85, 91, 66, 72]

lowest = min(marks)

print("Lowest:", lowest)
```

```
Lowest: 66
```

Prepared by: Ehtisham Rasheed

# Practical Mini Project — Student Marks Analyzer

```
marks = [78, 45, 89, 32, 91, 67, 55]

print("Number of students:", len(marks))

total = 0
passed = 0
failed = 0

for mark in marks:

    total += mark

    if mark >= 50:
        passed += 1
    else:
        failed += 1
```

```
average = total / len(marks)

print("Total marks:", total)
print("Average:", average)
print("Passed:", passed)
print("Failed:", failed)
print("Highest:", max(marks))
print("Lowest:", min(marks))
```

# Building a List from User Input

➢We can create a list dynamically.

```
students = []

for i in range(3):

    name = input("Enter student name: ")
    students.append(name)

print("Students:")

for student in students:
    print(student)
```

Pre n Rasheed

# In-Class Practice (Easy)

## ➢Exercise 1 — Create a List

- Create a list containing the names of five friends or classmates.
- Print the list.

## ➢Exercise 2 — Indexing

➢Given:

- numbers = [10, 20, 30, 40, 50]

➢Print:

- First element
- Last element
- Second element
- Second-last element

Prepared By: Ehtisham Rasheed

# In-Class Practice

- **Exercise 3 — Slicing**
- Given:
  - numbers = [10, 20, 30, 40, 50, 60]
- Print:
  - First three elements
  - Last three elements
  - Elements from index 2 to 4
  - Every second element
  - Reversed list

Prepared By: Ehtisham Rasheed

# In-Class Practice

- **Exercise 4 — Add Elements**
- Create:
  - students = ["Ali", "Ahmed"]
- Then:
  - Add "Sara" using append()
  - Add two more students using extend()
  - Insert a student at index 1

Prepared By: Ehtisham Rasheed

# In-Class Practice

- **Exercise 5 — Remove Elements**
- Given:
  - students = ["Ali", "Ahmed", "Sara", "Ayesha"]
- Try:
  - remove()
  - pop()
  - clear()

Prepared By: Ehtisham Rasheed

# In-Class Practice (Medium)

## ➢Exercise 6 — Student Marks

➢Given:

- marks = [45, 67, 89, 32, 76, 91, 55]

➢Calculate:

- Total
- Average
- Highest
- Lowest
- Number of passed students
- Number of failed students

Prepared By: Ehtisham Rasheed

# In-Class Practice (Medium)

- **Exercise 8 — Search**
- Given:
  - students = ["Ali", "Ahmed", "Sara", "Ayesha"]
- Ask the user for a name and check whether that student exists.
- **Exercise 9 — Sensor Data**
- Given:
  - sensor_data = [10, 15, -1, 20, 25, -1, 30]
  - Here -1 represents an invalid reading.
  - Create a new list containing only valid readings.
- Expected:
  - [10, 15, 20, 25, 30]

Prepared By: Ehtisham Rasheed